SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 4

NAME: P. DEEKSHANA

REG NO: 192425104

COURSE CODE: DSA0109

COURSE NAME: OBJECT ORIENTED PROGRAMMING WITH C++

TASK 1 – CLASS DESIGN REVIEW

1.1 Diagnose the design

The given classes have the following design problems:

1. **Repeated data members:** name, id, and basic are repeated in Professor and LabAssistant.
2. **Repeated pay calculation:** Both classes use the same DA calculation.
3. **Public data members:** Data can be changed directly without validation.
4. **Inconsistent names:** Professor uses name and id, while GuestFaculty uses nm and code.
5. **Unused field:** The da member is declared but never used.
6. **Magic number:** 0.42 is directly written in the calculation instead of being a named constant.
7. **Difficult extension:** Adding a new staff category requires modifying existing code.
8. **No common processing:** All staff cannot be processed easily through one base-class pointer loop.

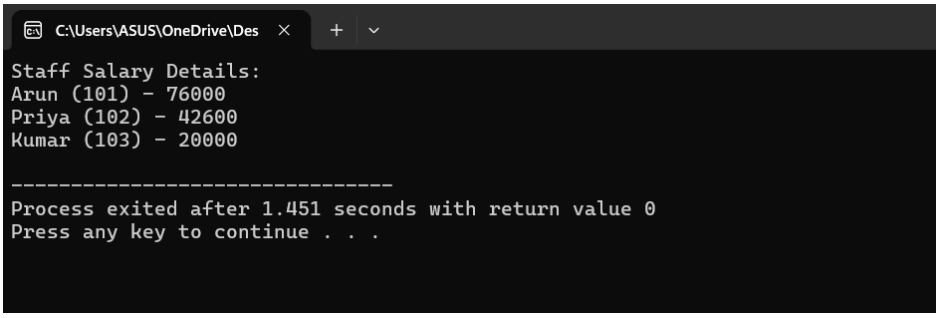
1.2 Repair the design

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

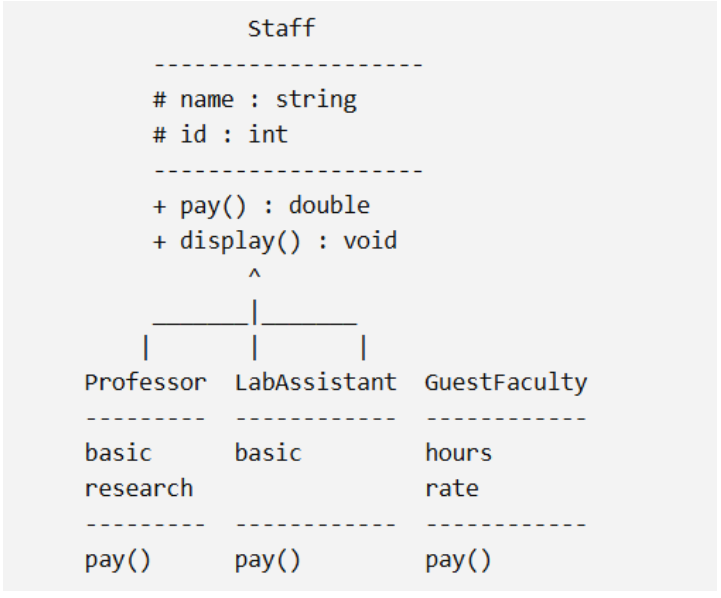
class Staff
{
protected:
    string name;
    int id;
public:
    Staff(string n, int i) : name(n), id(i) {}
    virtual double pay() const = 0;
    virtual void display() const
    {
        cout << name << " (" << id << ") - ";
        cout << pay() << endl;
    }
    virtual ~Staff() = default;
};
class Professor : public Staff
{
private:
    double basic, research;
    static constexpr double DA_RATE = 0.42;
public:
    Professor(string n, int i, double b, double r)
        : Staff(n, i), basic(b), research(r) {}
    double pay() const override
    {
        return basic + basic * DA_RATE + research;
    }
};
```

```cpp
class LabAssistant : public Staff
{
private:
    double basic;
    static constexpr double DA_RATE = 0.42;
public:
    LabAssistant(string n, int i, double b)
        : Staff(n, i), basic(b) {}
    double pay() const override
    {
        return basic + basic * DA_RATE;
    }
};
class GuestFaculty : public Staff
{
private:
    double hours, rate;
public:
    GuestFaculty(string n, int i, double h, double r)
        : Staff(n, i), hours(h), rate(r) {}
    double pay() const override
    {
        return hours * rate;
    }
};
int main()
{
    vector<Staff*> staff;
    staff.push_back(new Professor("Arun", 101, 50000, 5000));
    staff.push_back(new LabAssistant("Priya", 102, 30000));
    staff.push_back(new GuestFaculty("Kumar", 103, 40, 500));
    for (Staff* s : staff)
        s->display();
    for (Staff* s : staff)
        delete s;
    return 0;
}
```

Expected output:



UML class diagram:



1.3 Design defence

The hierarchy avoids repeated data and calculations by placing common information in the Staff base class. Each derived class implements its own pay() function. When a new staff category is added, we only create a new derived class and override pay(). Existing classes do not need to be modified. This makes the program reusable, extensible, and easier to maintain.

Alternative concept and complexity

If one Staff class uses a type field and a switch statement:

- Each pay() call takes O(1) time.
- Each object may contain unused fields for other staff types.
- Adding a new type requires changing the switch statement and possibly the data structure and related code.
- In inheritance, a new derived class can be added without changing existing classes.

TASK 2 – FROM REQUIREMENT TO HIERARCHY

2.1 Identify the relationships

Relationship	Type	Inheritance
Student is a Person	Is-a	Single
Staff is a Person	Is-a	Single
TeachingAssistant is a Student and Staff	Is-a	Multiple
ResearchScholar is a Student	Is-a	Single / Multilevel
Address belongs to Person	Has-a	Composition

The overall hierarchy uses hybrid inheritance because it combines multiple and hierarchical inheritance.

2.2 Resolve the diamond problem

If TeachingAssistant inherits from both Student and Staff, and both inherit from Person, TeachingAssistant receives two copies of Person.

This creates ambiguity when accessing Person members such as name.

Solution: Use virtual inheritance.

```
class Student : virtual public Person
{
};

class Staff : virtual public Person
{
};

class TeachingAssistant : public Student, public Staff
{
};
```

Virtual inheritance ensures that TeachingAssistant contains only one shared Person base-class object.

2.3 Design the classes

```
#include <iostream>
#include <string>
using namespace std;

class Person
{
protected:
    string name;
    int id;
    string phone;
public:
    Person(string n, int i, string p)
        : name(n), id(i), phone(p) {}
};
class Student : virtual public Person
{
protected:
    string programme;
    double cgpa;
public:
    Student(string n, int i, string p,
            string pr, double c)
        : Person(n, i, p), programme(pr), cgpa(c) {}
};
class Staff : virtual public Person
{
protected:
    string department;
    double salary;
public:
    Staff(string n, int i, string p,
          string d, double s)
        : Person(n, i, p), department(d), salary(s) {}
};
```

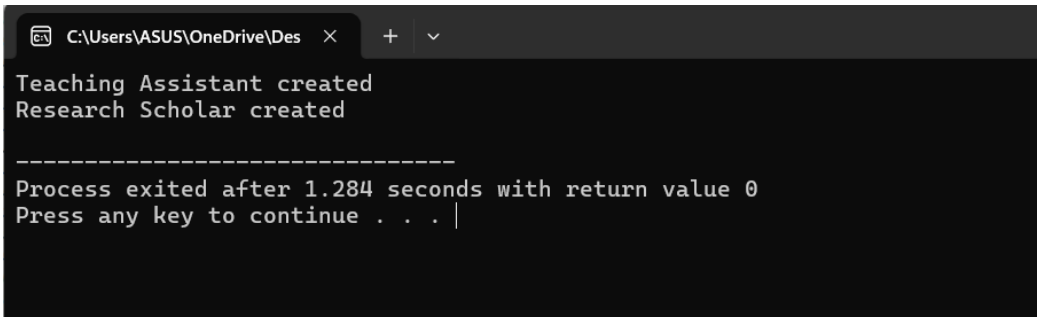
```
class TeachingAssistant : public Student, public Staff
{
public:
    TeachingAssistant(string n, int i, string p,
                      string pr, double c,
                      string d, double s)
        : Person(n, i, p),
          Student(n, i, p, pr, c),
          Staff(n, i, p, d, s) {}
};
class ResearchScholar : public Student
{
private:
    double fellowship;
public:
    ResearchScholar(string n, int i, string p,
                    string pr, double c, double f)
        : Person(n, i, p),
          Student(n, i, p, pr, c),
          fellowship(f) {}
};
int main()
{
    TeachingAssistant ta("Kavya", 101, "9876543210",
                         "AIML", 9.2, "CSE", 30000);
    ResearchScholar rs("Arun", 102, "9876501234",
                       "AIML", 9.5, 15000);
    cout << "Teaching Assistant created" << endl;
    cout << "Research Scholar created" << endl;
    return 0;
}
```

Expected output:

Teaching Assistant created

Research Scholar created

Output



2.4 Access table

Assume Person contains public, protected, and private members.

Person member	Public inheritance	Protected inheritance	Private inheritance
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not directly accessible	Not directly accessible	Not directly accessible

Predict the output

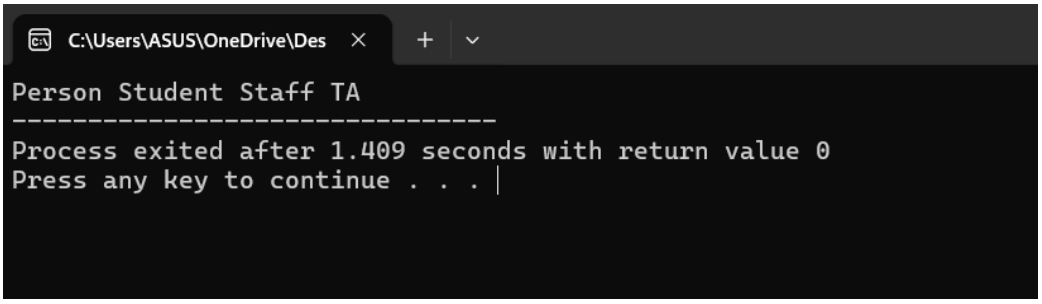
Given program:

```
#include <iostream>
using namespace std;
struct Person
{
    Person()
    {
        cout << "Person ";
    }
};
struct Student : virtual Person
{
    Student()
    {
        cout << "Student ";
    }
};
struct Staff : virtual Person
{
    Staff()
    {
        cout << "Staff ";
    }
};
struct TeachingAssistant : Student, Staff
{
    TeachingAssistant()
    {
        cout << "TA ";
    }
};
int main()
{
    TeachingAssistant ta;
    return 0;
}
```

Expected output:

Person Student Staff TA

Output



Explanation:

1. The virtual base class Person is constructed first.
2. Student is constructed next.
3. Staff is constructed after Student.
4. TeachingAssistant is constructed last.

Person is printed only once because virtual inheritance creates one shared Person sub-object.

If virtual is removed from both Student and Staff, the output becomes:

```
Person Student Person Staff TA
```

There will be two separate Person objects. This program still compiles because no ambiguous Person member is accessed. If we try to access ta.name, the compiler reports an ambiguity because there are two Person base-class sub-objects.

Alternative concept – Composition

Instead of inheriting from Student and Staff, TeachingAssistant could contain both as member objects.

```
class TeachingAssistant
{
    Student student;
    Staff staff;
};
```

- Composition represents a has-a relationship.
- It contains a Student object and a Staff object.
- Each member may contain its own Person sub-object.
- Accessing the name requires specifying student or staff.
- In virtual inheritance, there is one shared Person base sub-object.

Use sizeof(TeachingAssistant) to compare the actual object sizes on your compiler. The exact size depends on the compiler and its memory layout.

TASK 3 – CONSTRUCTORS IN DERIVED CLASSES AND MEMBER CLASSES

3.1 Pass values to base constructors

A derived-class constructor uses an initializer list to pass values to its base class.

```
class Student : virtual public Person
{
public:
    Student(string n, int i, string p)
        : Person(n, i, p) {}
};

class Staff : virtual public Person
{
public:
    Staff(string n, int i, string p)
        : Person(n, i, p) {}
};

class TeachingAssistant : public Student, public Staff
{
public:
    TeachingAssistant(string n, int i, string p)
        : Person(n, i, p),
          Student(n, i, p),
          Staff(n, i, p) {}
};
```

The most-derived class must initialize the virtual base class because it is responsible for constructing the shared virtual base. Initializer lists in intermediate classes are ignored for virtual bases when an object of the most-derived class is created.

3.2 Member classes

An Address object can be included inside Person, and a Course object can be included inside Student.

```
class Address
{
    string city;
    int pin;

public:
    Address(string c, int p) : city(c), pin(p) {}
};

class Course
{
    string courseName;

public:
    Course(string c) : courseName(c) {}
};
```

Example of using them as members:

```
class Person
{
protected:
    string name;
    Address address;

public:
    Person(string n, string city, int pin)
        : name(n), address(city, pin) {}
};

class Student : public Person
{
    Course course;

public:
    Student(string n, string city, int pin, string c)
        : Person(n, city, pin), course(c) {}
};
```

3.3 Trace construction and destruction

Program:

```
#include <iostream>
using namespace std;

class Person
{
public:
    Person() { cout << "Person "; }
    ~Person() { cout << "~Person "; }
};
class Student : virtual public Person
{
public:
    Student() { cout << "Student "; }
    ~Student() { cout << "~Student "; }
};
class Staff : virtual public Person
{
public:
    Staff() { cout << "Staff "; }
    ~Staff() { cout << "~Staff "; }
};
class TeachingAssistant : public Student, public Staff
{
public:
    TeachingAssistant() { cout << "TA "; }
    ~TeachingAssistant() { cout << "~TA "; }
};
int main()
{
    cout << "Student: ";
    {
        Student s;
    }
    cout << "\nStaff: ";
    {
        Staff st;
    }
    cout << "\nTeaching Assistant: ";
    {
        TeachingAssistant ta;
    }
    cout << endl;
    return 0;
}
```

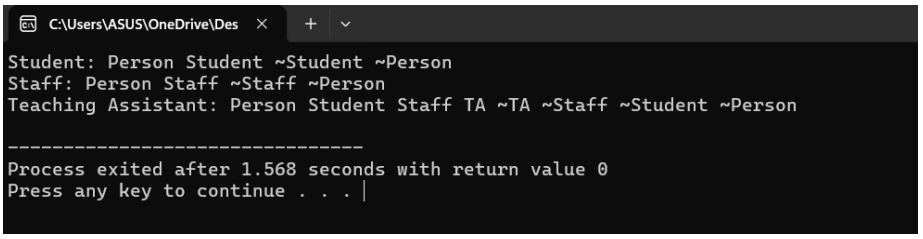
Expected output:

Student: Person Student ~Student ~Person

Staff: Person Staff ~Staff ~Person

Teaching Assistant: Person Student Staff TA ~TA ~Staff ~Student ~Person

Output



Rule:

- Base classes and member objects are constructed before the derived-class constructor body.
- The derived class is constructed last.
- Destruction happens in reverse order.
- Virtual base classes are constructed once.

3.4 Experiment

When virtual is removed, TeachingAssistant contains two Person sub-objects instead of one. Therefore, Person's constructor is called twice when creating a TeachingAssistant object. The size of the object also changes because of the additional Person sub-object. The exact size must be measured using sizeof(TeachingAssistant) on the compiler being used.

Predict the output

For the given Engine, Vehicle and Car program:

```
ʌɐɥʇᴉɔlɐ ǝuƃᴉuǝ cɐɹ | ~cɐɹ ~ǝuƃᴉuǝ ~ʌɐɥʇᴉɔlɐ
```

Explanation:

1. Vehicle is the base class, so it is constructed first.
2. Engine is a member object, so it is constructed next.
3. Car constructor runs last.
4. When the object goes out of scope, destruction happens in reverse order: Car, Engine, Vehicle.

Complete the code – TODO 1

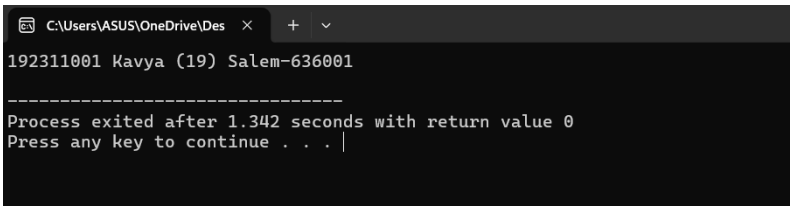
```
Student(const std::string& n, int a, long long r,
        const std::string& c, int p)
    : Person(n, a), regNo(r), home(c, p)
{
}
```

Complete program:

```cpp
#include <iostream>
#include <string>
using namespace std;

class Person
{
protected:
    string name;
    int age;
public:
    Person(string n, int a)
    {
        name = n;
        age = a;
    }
};
class Address
{
private:
    string city;
    int pin;
public:
    Address(string c, int p)
    {
        city = c;
        pin = p;
    }
    string str() const
    {
        return city + "-" + to_string(pin);
    }
};
class Student : public Person
{
private:
    long long regNo;
    Address home;
public:
    Student(string n, int a, long long r, string c, int p)
        : Person(n, a), regNo(r), home(c, p)
    {
    }
    void print() const
    {
        cout << regNo << " " << name << " ("
             << age << ") " << home.str() << endl;
    }
};
int main()
{
    Student s("Kavya", 19, 192311001, "Salem", 636001);
    s.print();
    return 0;
}
```

Output:



Initializer-list order experiment:

The actual initialization order follows the order in which members are declared in the class, not the order in which they appear in the initializer list. When the list order is changed, -Wall -Wextra can show a -Wreorder warning.

Alternative concept – Pointer to Address

If Address is stored using a pointer:

```
Address* home;
```

The constructor must allocate memory:

```
home = new Address(c, p);
```

The destructor must release it:

```
delete home;
```

The copy constructor and assignment operator must perform a deep copy to avoid two objects sharing the same dynamically allocated Address.

Complexity comparison:

Feature	Address member object	Address pointer
Memory allocation	Automatic	Dynamic
Allocation time	No separate allocation	O(1) allocation
Space	Stores Address directly	Stores pointer plus allocated Address
Destructor	Automatic	Must use delete
Copying	Compiler-generated copy is safe here	Deep copy is required

---

TASK 4 – POINTERS TO DERIVED CLASSES

4.1 Locate the defects

1. rate() is not virtual, so calling it through an Account* uses the base-class version.
2. The base-class destructor is not virtual. Deleting a SeniorAccount through an Account pointer causes undefined behavior.
3. The dynamically allocated history array may not be released correctly because of the non-virtual destructor.
4. Account copy = SeniorAccount(); causes object slicing. The derived part of SeniorAccount is removed, and only the Account portion is copied.

4.2 Construct evidence

Add messages to the destructors:

```
class Account
{
public:
    double rate() const { return 3.5; }

    ~Account()
    {
        cout << "Account destructor\n";
    }
};

class SeniorAccount : public Account
{
    double* history = new double[100];

public:
    double rate() const { return 4.0; }

    ~SeniorAccount()
    {
        cout << "SeniorAccount destructor\n";
        delete[] history;
    }
};
```

With a non-virtual base destructor, deleting a derived object through a base pointer has undefined behavior. The derived destructor may not run, and the allocated array may leak.

To check memory errors, compile using:

```
g++ -std=c++17 -Wall -Wextra -fsanitize=address -g task4.cpp -o task4
```

Run:

```
./task4
```

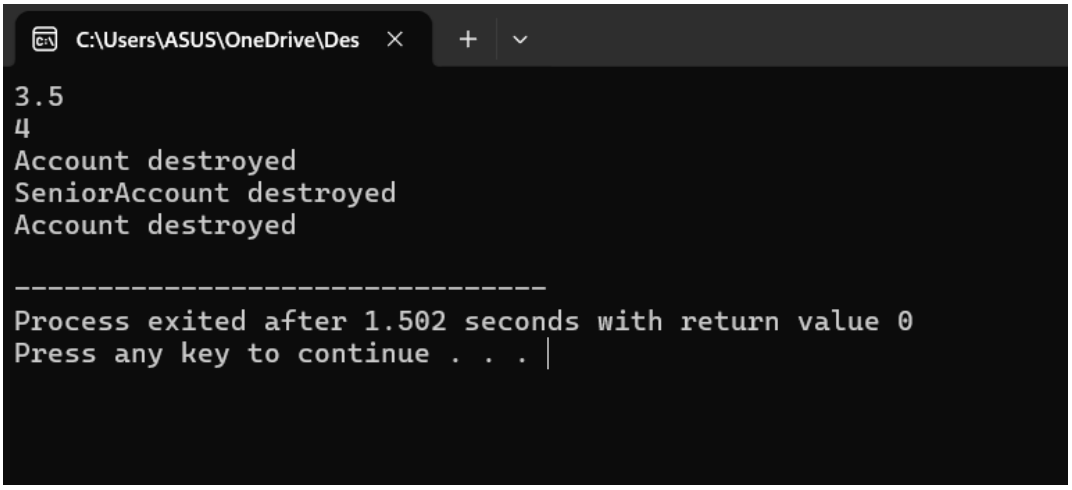
The actual sanitizer report depends on the compiler and runtime. Do not write an assumed sanitizer result as an actual experiment.

4.3 Repair the program

```
#include <iostream>
#include <vector>
using namespace std;
class Account
{
public:
    virtual double rate() const
    {
        return 3.5;
    }
    virtual ~Account()
    {
        cout << "Account destroyed" << endl;
    }
};
class SeniorAccount : public Account
{
private:
    double* history;
public:
    SeniorAccount()
    {
        history = new double[100];
    }
    double rate() const
    {
        return 4.0;
    }
    ~SeniorAccount()
    {
        delete[] history;
        cout << "SeniorAccount destroyed" << endl;
    }
```

```
};
int main()
{
    vector<Account*> accounts;
    accounts.push_back(new Account());
    accounts.push_back(new SeniorAccount());
    for (int i = 0; i < accounts.size(); i++)
    {
        cout << accounts[i]->rate() << endl;
    }
    for (int i = 0; i < accounts.size(); i++)
    {
        delete accounts[i];
    }
    return 0;
}
```

Expected output:



The last two lines are produced when the objects are destroyed. The derived destructor runs first, followed by the base destructor.

4.4 Prevent recurrence

1. A base class used polymorphically should have a virtual destructor.
2. Use the override keyword when overriding virtual functions to detect mistakes at compile time.

Predict the output

Given:

```
Account* p = &s;
Account copy = s;

cout << p->rate() << ' '
     << p->vrate() << ' '
     << copy.vrate() << '\n';
```

Expected output:

```
3.5 4 3.5
```

Explanation:

- p->rate() prints 3.5 because rate() is not virtual. The call is resolved using the pointer's static type, Account.

- p->vrate() prints 4 because vrate() is virtual. The actual object is SeniorAccount, so the derived version runs.
- copy.vrate() prints 3.5 because copying SeniorAccount into an Account object causes object slicing.

Alternative concept – std::unique_ptr

Using std::unique_ptr<Account> inside a vector automatically releases memory when the vector is destroyed.

- Processing n accounts takes O(n) time.
- Each account still needs its object memory and a pointer-like smart pointer.
- It prevents ordinary memory leaks caused by forgetting delete.
- It does not automatically prevent incorrect non-virtual function calls or object slicing if objects are copied by value.
- A virtual destructor is still important for correct polymorphic destruction.

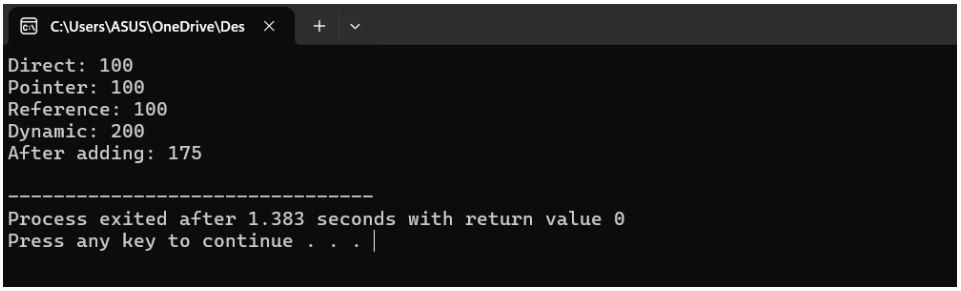
TASK 5 – POINTERS TO OBJECTS AND THE this POINTER

5.1 Four ways to reach an object

```cpp
#include <iostream>
using namespace std;

class Cart
{
    double amount;
public:
    Cart(double amount)
    {
        this->amount = amount;
    }
    Cart& add(double amount)
    {
        this->amount += amount;
        return *this;
    }
    double total() const
    {
        return amount;
    }
};
int main()
{
    // 1. Directly by object name
    Cart c(100);
    cout << "Direct: " << c.total() << endl;
    // 2. Through pointer
    Cart* p = &c;
    cout << "Pointer: " << p->total() << endl;
    // 3. Through reference
    Cart& r = c;
    cout << "Reference: " << r.total() << endl;
    // 4. Dynamic object using new and delete
    Cart* d = new Cart(200);
    cout << "Dynamic: " << d->total() << endl;
    delete d;
    // Method chaining
    c.add(50).add(25);
    cout << "After adding: " << c.total() << endl;
    return 0;
}
```

Expected output:



5.2 Array of pointers to Cart objects

The following code finds the cart with the highest total.

```
#include <iostream>
using namespace std;

class Cart
{
    double amount;

public:
    Cart(double a) : amount(a) {}

    double total() const
    {
        return amount;
    }
};

int main()
{
    Cart c1(500), c2(800), c3(600);

    Cart* carts[3] = {&c1, &c2, &c3};

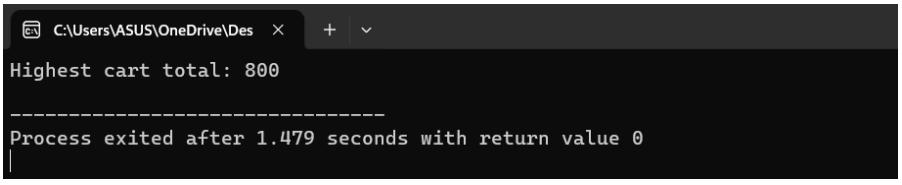
    Cart* highest = carts[0];

    for (int i = 1; i < 3; i++)
    {
        if (carts[i]->total() > highest->total())
            highest = carts[i];
    }

    cout << "Highest cart total: "
         << highest->total() << endl;

    return 0;
}
```

Output:



5.3 Comparison

Method	Safety	Lifetime	Usage
Direct object	Safe	Depends on scope	Normal object access
Pointer	Can be null	Does not own object	Access through address

		automatically	
Reference	Cannot normally be null	Depends on referred object	Alternative name for object
new and delete	Requires careful memory management	Manually controlled	Dynamic object creation

What is this pointer?

- this is a pointer that refers to the current object.
- *this refers to the current object itself.
- this->amount accesses the member variable of the current object.
- Returning *this by reference allows method chaining.

Complete the code – TODO 1 and TODO 2

```
Cart& add(const string& item, double price)
{
    if (n < 10)
    {
        items[n] = item;
        prices[n] = price;
        n++;
    }

    return *this;
}

Cart& setDiscount(double discount)
{
    this->discount = discount;
    return *this;
}
```

Complete program:

```
#include <iomanip>
#include <iostream>
#include <string>
using namespace std;

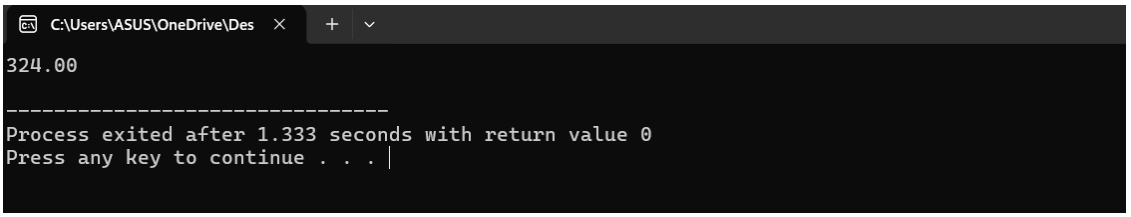
class Cart
{
    string items[10];
    double prices[10];
    int n = 0;
    double discount = 0;

public:
    Cart& add(const string& item, double price)
    {
        if (n < 10)
        {
            items[n] = item;
            prices[n] = price;
            n++;
        }

        return *this;
    }
```

```
    Cart& setDiscount(double discount)
    {
        this->discount = discount;
        return *this;
    }
    double total() const
    {
        double t = 0;
        for (int i = 0; i < n; ++i)
            t += prices[i];
        return t * (100 - discount) / 100;
    }
};
int main()
{
    Cart c;
    c.add("Rice", 60).add("Dal", 120).setDiscount(10);
    Cart* p = &c;
    p->add("Oil", 180);
    cout << fixed << setprecision(2)
         << p->total() << '\n';
    return 0;
}
```

Output:



Explanation:

Total amount = 60 + 120 + 180 = 360.

Discount = 10%.

Final amount = 360 × 90 / 100 = 324.

What happens if add() returns Cart instead of Cart&?

If add() returns Cart instead of Cart&, it returns a copy of the object. Method chaining may still work, but each call can create temporary copies. Returning Cart& avoids unnecessary copying and allows operations on the original object.

Alternative concept – std::vector<Cart>

If carts are stored as objects in a vector:

```
vector<Cart> carts;
carts.push_back(Cart());
```

- Finding the highest total takes O(n) time in both designs.
- A vector of objects stores complete Cart objects.
- An array of pointers stores only addresses in the array, while the actual objects are stored elsewhere.
- When a vector grows, it may reallocate its memory. Pointers and references to its elements can become invalid.
- A vector of objects is generally simpler when the objects do not require polymorphism.

TASK 6 – DESIGN CHALLENGE: VEHICLE RENTAL FLEET

Stage 1 – Interpret the problem

Entities:

1. Vehicle
2. Bicycle
3. TwoWheeler
4. Car
5. ElectricCar

Common data:

- Registration number
- Daily rental rate
- Rental status

Different data:

- Motor vehicles have fuel or charge level and service due date.
- Cars have seating capacity.
- Electric cars have battery capacity and charging cost per kWh.

Different behaviour:

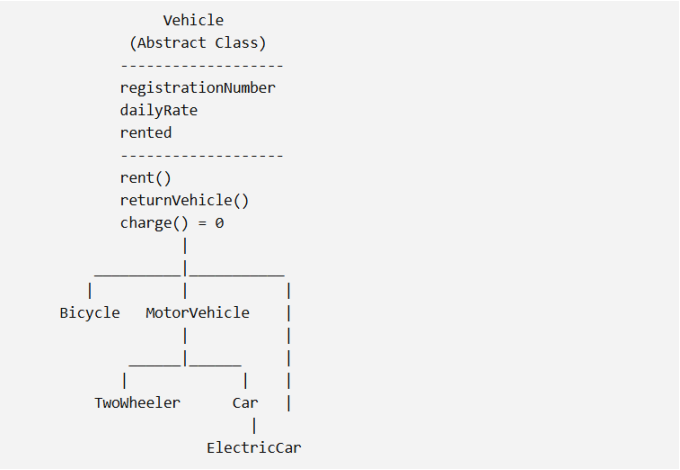
Each vehicle calculates its rental charge differently.

Testable requirements:

1. The system must add different vehicle types.
2. It must display all available vehicles , It must allow a vehicle to be rented.
3. It must prevent an already rented vehicle from being rented again.
4. It must allow a rented vehicle to be returned , It must calculate late charges.
5. It must support adding new vehicle types without modifying existing vehicle classes.

Stage 2 – Class hierarchy

Use an abstract base class named Vehicle.



Inheritance types:

- Vehicle to Bicycle: Single inheritance.
- Vehicle to MotorVehicle: Single inheritance.
- MotorVehicle to TwoWheeler and Car: Hierarchical inheritance.
- Car to ElectricCar: Multilevel inheritance.

Stage 3 – Critical design decisions

1. Common data and functions are placed in Vehicle.
2. charge() is declared as a pure virtual function.
3. Derived classes override charge().
4. rent() and returnVehicle() manage the rental status.
5. The fleet is stored using an array or vector of base-class pointers.
6. A virtual destructor ensures correct destruction of derived objects.

Stage 4 and Stage 5 – Class implementation and demonstration

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
class Vehicle
{
protected:
    string regNo;
    double dailyRate;
    bool rented;
public:
    Vehicle(string r, double rate)
    {
        regNo = r;
        dailyRate = rate;
        rented = false;
    }
    virtual double charge(int days) const = 0;
    void rent()
    {
        if (!rented)
        {
            rented = true;
            cout << regNo << " rented successfully" << endl;
        }
        else
        {
            cout << regNo << " is already rented" << endl;
        }
    }
    void returnVehicle(int daysLate)
    {
        if (rented)
        {
            rented = false;
            double lateFee = daysLate * 200;
            cout << regNo << " returned successfully" << endl;
            cout << "Late charge: " << lateFee << endl;
        }
        else
        {
            cout << regNo << " was not rented" << endl;
        }
    }
```

```
    virtual void display() const
    {
        cout << "Registration: " << regNo
             << ", Daily rate: " << dailyRate << endl;
    }
    virtual ~Vehicle() {}
};
class Bicycle : public Vehicle
{
public:
    Bicycle(string r, double rate)
        : Vehicle(r, rate) {}
    double charge(int days) const
    {
        return dailyRate * days;
    }
};
class MotorVehicle : public Vehicle
{
protected:
    double fuelLevel;
    string serviceDate;
public:
    MotorVehicle(string r, double rate,
                 double fuel, string date)
        : Vehicle(r, rate)
    {
        fuelLevel = fuel;
        serviceDate = date;
    }
};
class TwoWheeler : public MotorVehicle
{
public:
    TwoWheeler(string r, double rate,
               double fuel, string date)
        : MotorVehicle(r, rate, fuel, date) {}
    double charge(int days) const
    {
        return dailyRate * days;
    }
};
```

```
};

class Car : public MotorVehicle
{
protected:
    int seatingCapacity;
public:
    Car(string r, double rate, double fuel,
        string date, int seats)
        : MotorVehicle(r, rate, fuel, date)
    {
        seatingCapacity = seats;
    }
    double charge(int days) const
    {
        return dailyRate * days;
    }
};
class ElectricCar : public Car
{
private:
    double batteryCapacity;
    double costPerKWh;
public:
    ElectricCar(string r, double rate, double fuel,
                string date, int seats,
                double battery, double cost)
        : Car(r, rate, fuel, date, seats)
    {
        batteryCapacity = battery;
        costPerKWh = cost;
    }
    double charge(int days) const
    {
        double rental = dailyRate * days;
        double chargingCost = batteryCapacity * costPerKWh;

        return rental + chargingCost;
    }
};
class ElectricBus : public Vehicle
{
```

```
    : Vehicle(r, rate)
    {
        batteryCapacity = battery;
        costPerKWh = cost;
    }
    double charge(int days) const
    {
        return dailyRate * days +
            batteryCapacity * costPerKWh;
    }
};
int main()
{
    vector<Vehicle*> fleet;
    fleet.push_back(new Bicycle("B101", 100));
    fleet.push_back(new TwoWheeler(
        "T201", 300, 80, "2026-12-01"));
    fleet.push_back(new Car(
        "C301", 1000, 60, "2026-12-05", 5));
    fleet.push_back(new ElectricCar(
        "E401", 1500, 0, "2026-12-10",
        5, 50, 8));
    fleet.push_back(new ElectricBus(
        "EB501", 2500, 100, 8));
    cout << "VEHICLE RENTAL FLEET" << endl;
    for (int i = 0; i < fleet.size(); i++)
    {
        fleet[i]->display();
    }
    cout << "\nRENTAL DETAILS" << endl;
    for (int i = 0; i < fleet.size(); i++)
    {
        fleet[i]->rent();
        cout << "Rental charge for 2 days: "
            << fleet[i]->charge(2) << endl;
    }
    cout << "\nRETURN DETAILS" << endl;
    fleet[0]->returnVehicle(1);
    fleet[1]->returnVehicle(0);
    fleet[2]->returnVehicle(2);
    fleet[3]->returnVehicle(0);
    fleet[4]->returnVehicle(0);
    for (int i = 0; i < fleet.size(); i++)
    {
        delete fleet[i];
    }
    return 0;
}
```

Expected output:

```
C:\Users\ASUS\OneDrive\Des
VEHICLE RENTAL FLEET
Registration: B101, Daily rate: 100
Registration: T201, Daily rate: 300
Registration: C301, Daily rate: 1000
Registration: E401, Daily rate: 1500
Registration: EB501, Daily rate: 2500

RENTAL DETAILS
B101 rented successfully
Rental charge for 2 days: 200
T201 rented successfully
Rental charge for 2 days: 600
C301 rented successfully
Rental charge for 2 days: 2000
E401 rented successfully
Rental charge for 2 days: 3400
EB501 rented successfully
Rental charge for 2 days: 5800

RETURN DETAILS
B101 returned successfully
Late charge: 200
T201 returned successfully
Late charge: 0
C301 returned successfully
Late charge: 400
E401 returned successfully
Late charge: 0
EB501 returned successfully
Late charge: 0

--------------------------------
Process exited after 1.362 seconds with return value 0
Press any key to continue . . .
```

Pseudocode for renting a vehicle

```
ALGORITHM RentVehicle(vehicle)

BEGIN
    IF vehicle is not rented THEN
        Set rented status to TRUE
        Display "Vehicle rented successfully"
    ELSE
        Display "Vehicle is already rented"
    END IF
END
```

Pseudocode for returning a vehicle

```
ALGORITHM ReturnVehicle(vehicle, lateDays)

BEGIN
    IF vehicle is rented THEN
        Set rented status to FALSE
        Calculate lateFee = lateDays * 200
        Display return confirmation
        Display lateFee
    ELSE
        Display "Vehicle was not rented"
    END IF
END
```

Adding a new vehicle type

The program adds ElectricBus by inheriting from Vehicle and implementing the charge() function.

The existing Vehicle, Bicycle, Car, and other classes do not need to be changed. This demonstrates extensibility and code reusability.

Alternative concept – Single Vehicle class with switch

Instead of inheritance, a single Vehicle class could use a type code.

```
double charge(int days)
{
    switch (type)
    {
        case 1:
            return dailyRate * days;

        case 2:
            return dailyRate * days;

        case 3:
            return dailyRate * days;

        case 4:
            return dailyRate * days +
                   batteryCapacity * costPerKWh;

        default:
            return 0;
    }
}
```

Complexity comparison:

Feature	Single class with switch	Inheritance with virtual functions
Time per charge call	O(1)	O(1)
Space per object	May contain unused fields	Base data plus derived data and typically a vtable pointer
Adding a new vehicle	Modify switch and related code	Add a new derived class
Maintenance	More difficult as types increase	Easier to extend
Reusability	Limited	High